

# Análisis técnico de CockroachDB

Distribución de datos, consistencia CAP y pertinencia para TiendaTech

**Autor:** Jeremy Ruperto Gaibor Rodríguez

**Actividad:** FORM-IC-02

**Asignatura:** Aplicaciones Distribuidas

**Proyecto:** TiendaTech

**Síntesis.** CockroachDB es una base de datos *Distributed SQL* que conserva transacciones relacionales mientras distribuye y replica la información entre varios nodos. Este análisis revisa su organización de datos, su clasificación según CAP, un caso real y su posible aplicación en TiendaTech.

## 1. Introducción

CockroachDB conserva tablas, consultas SQL y transacciones ACID, pero distribuye la información entre varios nodos. Para la aplicación se presenta como una sola base lógica, mientras el motor administra la ubicación y las réplicas. En despliegues multirregión también permite declarar mediante SQL la localidad de los datos y los objetivos de supervivencia [1], [2].

Su utilidad aparece cuando una aplicación necesita operar en distintas ubicaciones sin mantener una base independiente por cada sede. En un sistema pequeño, esa complejidad puede no estar justificada.

## 2. Distribución de datos

Las tablas e índices se representan en un espacio ordenado de claves dividido en unidades llamadas *rangos*. Estos pueden separarse al crecer, repartirse entre nodos y rebalancearse cuando cambia la composición del clúster [2], [3]. Desde la aplicación no es necesario conocer qué servidor almacena cada fila.

Cada rango mantiene varias réplicas coordinadas mediante Raft. Una escritura solo se confirma cuando existe acuerdo de un quórum. Con tres réplicas se requieren dos; si se pierde esa mayoría, el rango afectado deja de aceptar operaciones que necesitan consenso [4], [5].

Cuadro 1: Modalidades de localidad en CockroachDB

| Configuración     | Uso principal                                      |
|-------------------|----------------------------------------------------|
| GLOBAL            | Favorece lecturas desde distintas regiones.        |
| REGIONAL BY TABLE | Mantiene la tabla cerca de una región principal.   |
| REGIONAL BY ROW   | Permite asignar cada fila a una región específica. |

La modalidad debe elegirse según el patrón de lectura y escritura de cada tabla, no aplicarse de igual forma a toda la base [1], [2].

## 3. Consistencia según CAP

CockroachDB puede analizarse principalmente como un sistema **CP**. Durante una partición mantiene la consistencia y la tolerancia a particiones; el grupo sin mayoría no puede confirmar escrituras. La disponibilidad puede reducirse temporalmente antes que permitir estados incompatibles [4], [6].

La capa transaccional admite operaciones ACID sobre diferentes filas, tablas y rangos. Bajo aislamiento serializable, las transacciones concurrentes deben producir un resultado equivalente a una ejecución ordenada [1], [6]. En TiendaTech, esta propiedad permitiría registrar un pedido y reducir el inventario dentro de una sola transacción.

## 4. Caso real documentado

Hard Rock Digital utiliza CockroachDB en una plataforma de apuestas deportivas que opera en varios estados de Estados Unidos. La solución mantiene una sola base lógica y combina regiones de AWS, zonas locales y AWS Outposts para cumplir requisitos de disponibilidad, escalamiento y residencia de datos [5], [7].

El caso reporta operación en ocho estados o regiones y cerca de cien nodos durante periodos de

alta demanda. Las fuentes pertenecen a Cockroach Labs, por lo que son válidas para documentar la implementación, pero no constituyen una comparación independiente con otras tecnologías.

## 5. Pertinencia para TiendaTech

En TiendaTech, productos, marcas y categorías podrían evaluarse como información global, porque todas las sucursales consultarían el mismo catálogo. El inventario podría manejarse como regional por fila, asociando cada existencia con la sucursal responsable [1], [2].

Los pedidos y la reducción del stock deberían ejecutarse dentro de una sola transacción. Así se registraría la venta y se descontaría la existencia, o se cancelarían ambas operaciones si ocurre un error.

**Decisión técnica.** Mantendría PostgreSQL y `postgres_fdw` en la versión académica actual, porque permiten demostrar explícitamente la fragmentación, la replicación y la reconstrucción. Consideraría CockroachDB si TiendaTech operara en varias sedes reales, con alta concurrencia y necesidad de continuar ante fallos.

## 6. Conclusión

CockroachDB distribuye la información mediante rangos replicados y coordinados por Raft. Su comportamiento es principalmente CP porque exige quórum y prioriza la consistencia durante una partición. Hard Rock Digital demuestra que puede utilizarse en un entorno multirregión real.

Para TiendaTech sería una alternativa futura. En la implementación actual, PostgreSQL sigue siendo suficiente y permite comprender de forma más directa los mecanismos de distribución exigidos en la asignatura.

## Referencias

- [1] N. VanBenschoten et al., «Enabling the Next Generation of Multi-Region Applications with CockroachDB», en *Proceedings of the 2022 International Conference on Management of Data (SIGMOD '22)*, Philadelphia, PA, USA: Association for Computing Machinery, 2022, págs. 2312-2325. DOI: [10.1145/3514221.3526053](https://doi.org/10.1145/3514221.3526053).
- [2] A. Ajmani et al., «A Demonstration of Multi-Region CockroachDB», *Proceedings of the VLDB Endowment*, vol. 15, n.º 12, págs. 3610-3613, 2022. DOI: [10.14778/3554821.3554856](https://doi.org/10.14778/3554821.3554856).
- [3] Cockroach Labs. «Distribution Layer». CockroachDB Documentation, visitado 7 de jul. de 2026. dirección: <https://www.cockroachlabs.com/docs/stable/architecture/distribution-layer>.
- [4] Cockroach Labs. «Replication Layer». CockroachDB Documentation, visitado 7 de jul. de 2026. dirección: <https://www.cockroachlabs.com/docs/stable/architecture/replication-layer>.
- [5] Cockroach Labs. «How Hard Rock Digital Is Using CockroachDB to Become a Top Sports Betting and Interactive Gaming Platform». Customer case study, visitado 7 de jul. de 2026. dirección: <https://www.cockroachlabs.com/customers/hard-rock-digital/>.
- [6] Cockroach Labs. «Transaction Layer». CockroachDB Documentation, visitado 7 de jul. de 2026. dirección: <https://www.cockroachlabs.com/docs/stable/architecture/transaction-layer>.
- [7] C. McAllister. «How Hard Rock Digital Built a Highly Available and Compliant Sports Betting App», visitado 7 de jul. de 2026. dirección: <https://www.cockroachlabs.com/blog/highly-available-sports-betting-app/>.